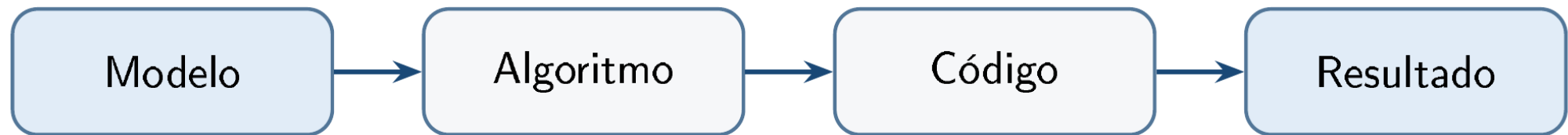


Conceptos Fundamentales en Programación

Variables, entrada/salida y operaciones en FORTRAN



Física Computacional – 106018C

Departamento de Física · Universidad del Valle

Semestre 2026-II

Objetivos de aprendizaje

- Explicar qué significa **programar** y reconocer los componentes básicos de un programa.
- Interpretar y construir **diagramas de flujo** sencillos.
- Reconocer variables y tipos fundamentales en **FORTRAN**.
- Usar `read` y `print` para entrada y salida.
- Aplicar operadores matemáticos, relacionales y lógicos.
- Adoptar buenas prácticas desde el primer programa.

Idea central

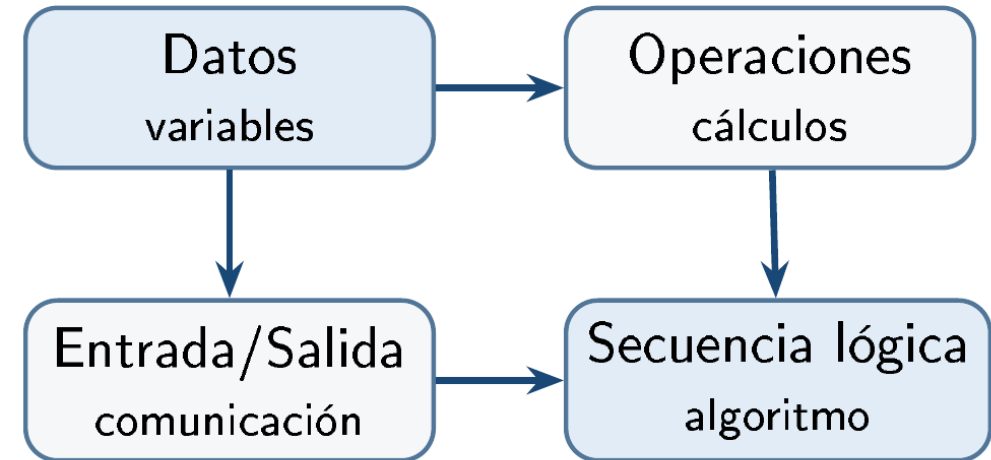
Antes de escribir código, debemos entender el problema y diseñar una secuencia lógica de pasos.

¿Qué es programar?

Programar significa expresar una solución mediante instrucciones que el computador puede ejecutar. Un programa combina

cuatro componentes fundamentales:

- datos,
- operaciones,
- decisiones,
- entrada y salida.

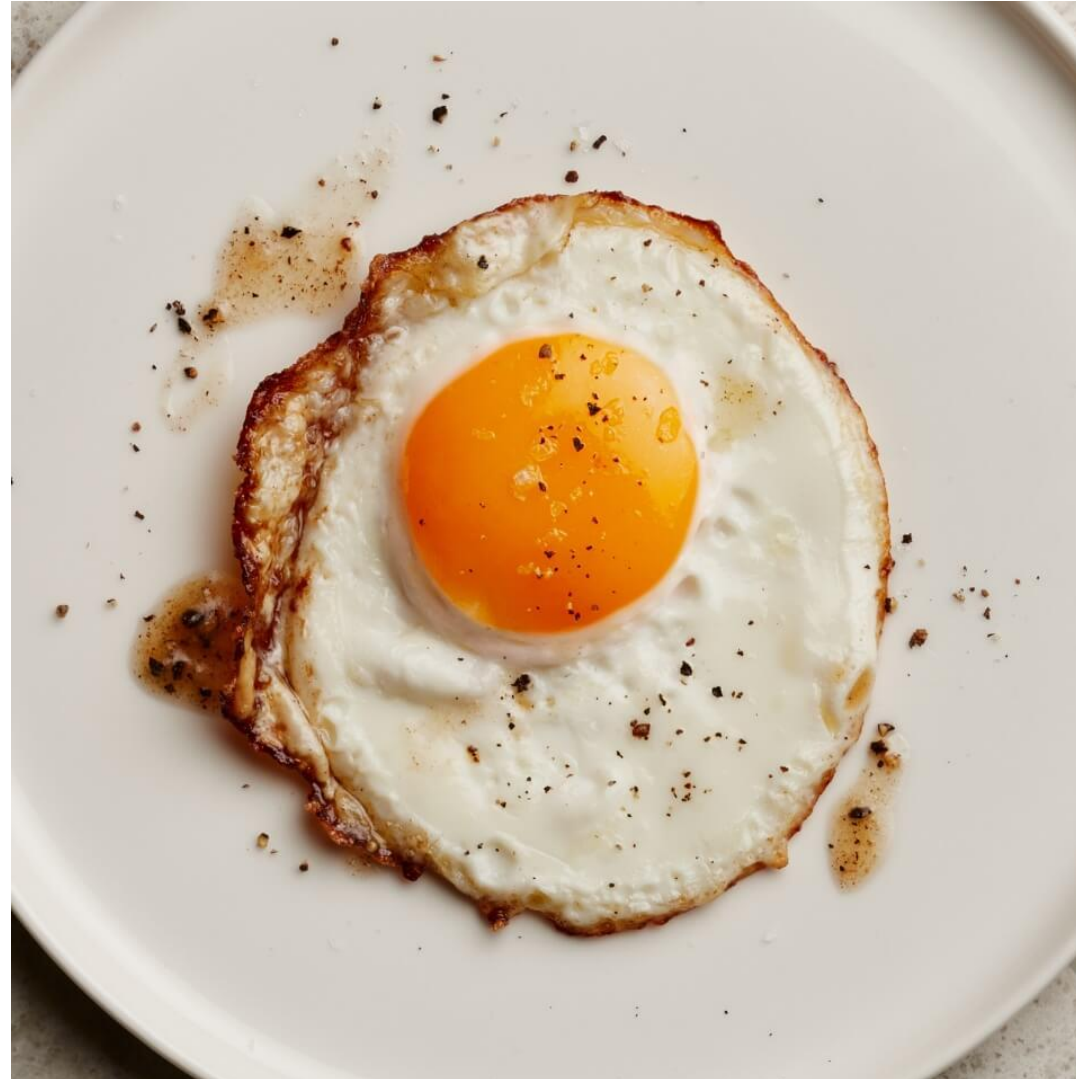


Primero diseñamos la solución; después la traducimos a FORTRAN.

Símbolos básicos de un diagrama de flujo

Símbolo	Nombre	Uso
	Terminal	Marca el inicio o el fin de un programa.
	Entrada/Salida	Lee información o muestra resultados.
	Proceso	Realiza asignaciones u operaciones.
	Decisión	Cambia el flujo según una condición.

Como freir un huevo?



Freir un huevo - pseudocodigo

INICIO

Tomar una sartén

Agregar aceite

Encender la estufa

Calentar la sartén

Romper el huevo en la sartén

MIENTRAS el huevo no esté suficientemente cocido

HACER

Continuar cocinando

FIN MIENTRAS

Apagar la estufa

Servir el huevo

Freir un huevo - flowchart

flowchart TD

A([Inicio])

B[\"Tomar una sartén\"]

C[\"Agregar aceite\"]

D[\"Encender la estufa\"]

E[\"Calentar la sartén\"]

F[\"Romper el huevo en la sartén\"]

G{\"¿El huevo está suficientemente cocido?\"}

H[\"Continuar cocinando\"]

I[\"Apagar la estufa\"]

J[\"Servir el huevo\"]

K([Fin])

A --> B

B --> C

C --> D

D --> E

E --> F

F --> G

G -- \"No\" --> H

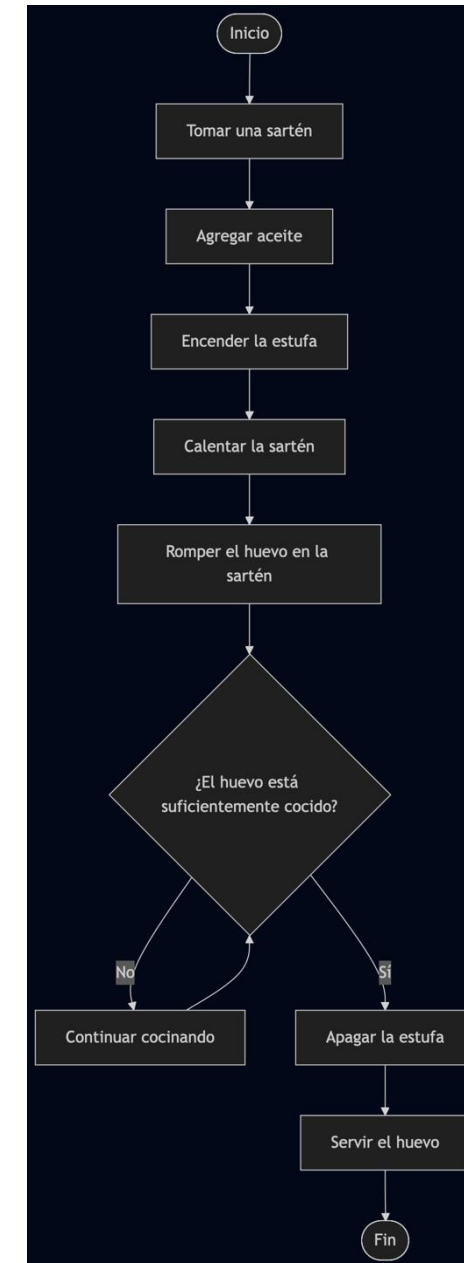
H --> G

G -- \"Sí\" --> I

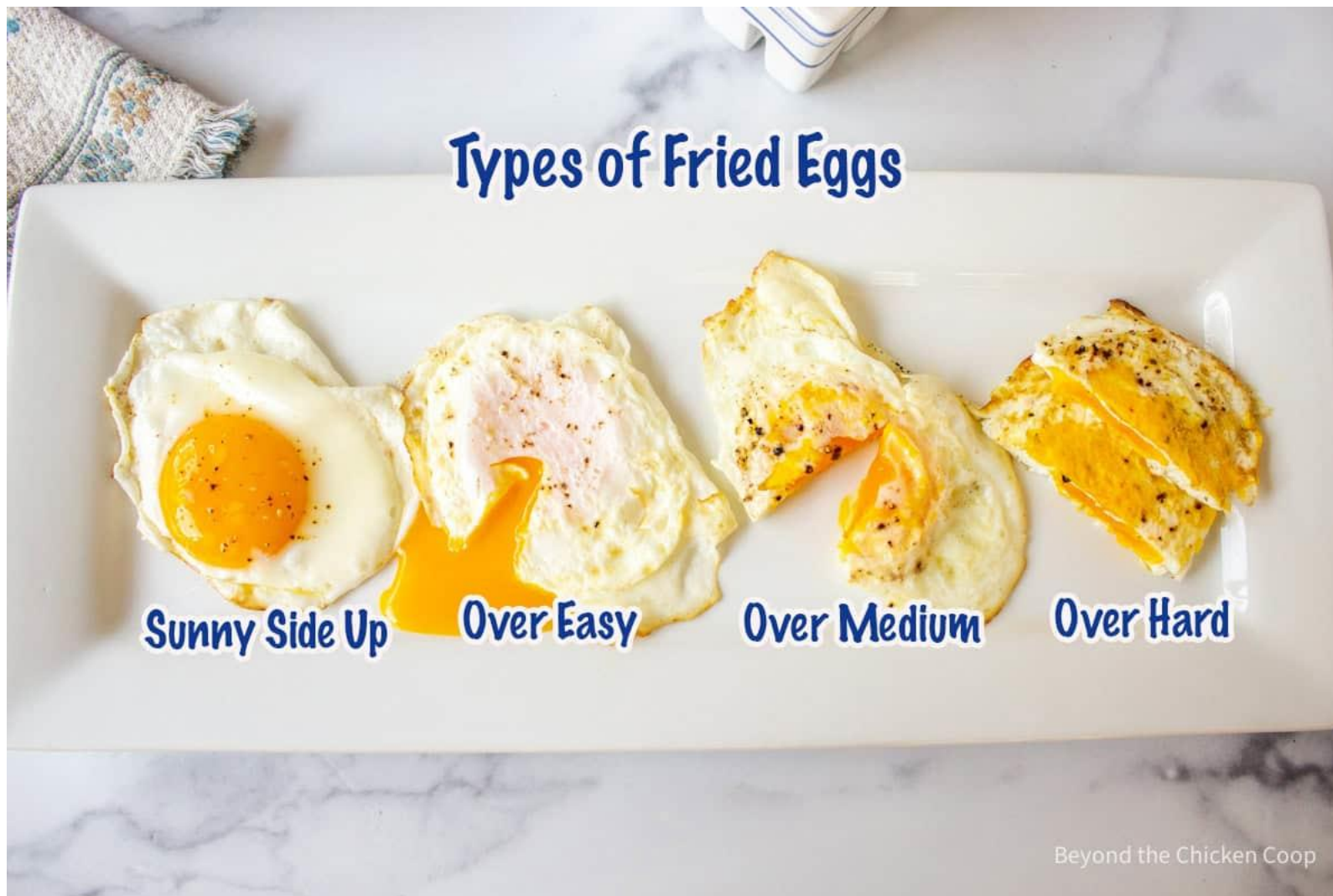
I --> J

J --> K

<https://mermaid.live/>



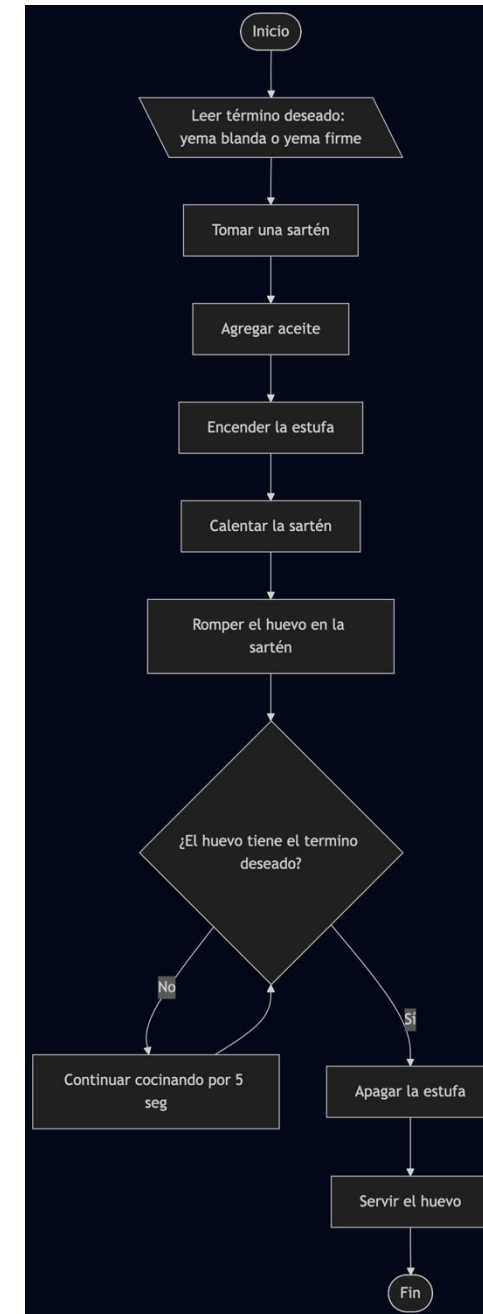
Y si yo quiero el huevo blando o duro?



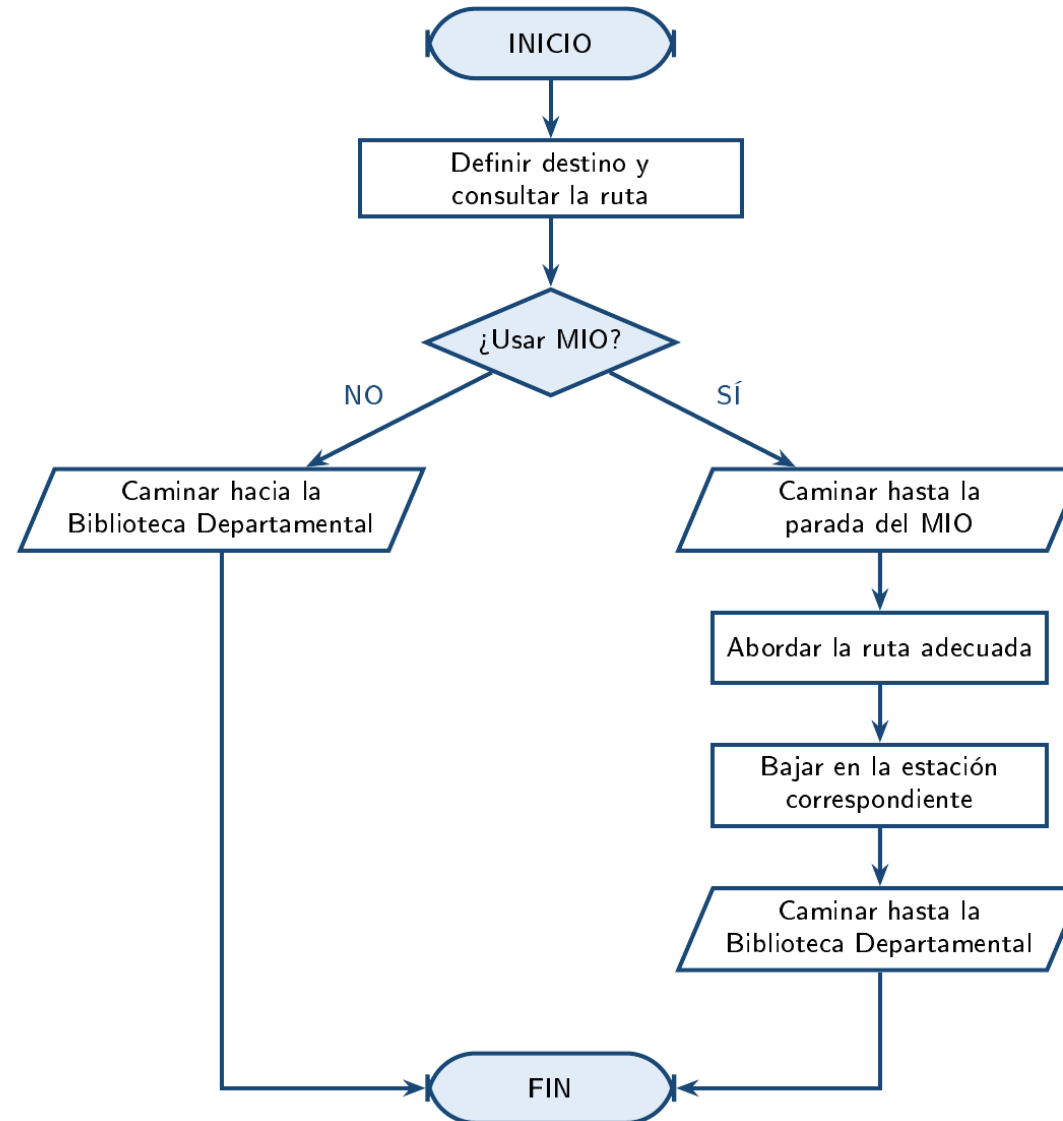
Freir un huevo al gusto

- flowchart TD
- A([Inicio])
- B1[/Leer término deseado: yema blanda o yema firme/]
- B[Tomar una sartén]
- C[Agregar aceite]
- D[Encender la estufa]
- E[Calentar la sartén]
- F[Romper el huevo en la sartén]
- G{"¿El huevo tiene el termino deseado?"}
- H[Continuar cocinando por 5 seg"]
- I[Apagar la estufa]
- J[Servir el huevo]
- K([Fin])
-

- A --> B1
- B1-->B
- B --> C
- C --> D
- D --> E
- E --> F
- F --> G
- G -- "No" --> H
- H --> G
- G -- "Sí" --> I
- I --> J
- J --> K



Ejercicio 1: ir a la Biblioteca Departamental



Variables y tipos de datos

Toda variable tiene

- un **nombre**,
- un **tipo**,
- un **valor**.

En FORTRAN se recomienda
usar siempre:

```
implicit none
```

para obligar a declarar todas
las variables.

integer
enteros

real
decimales

logical
verdadero/falso

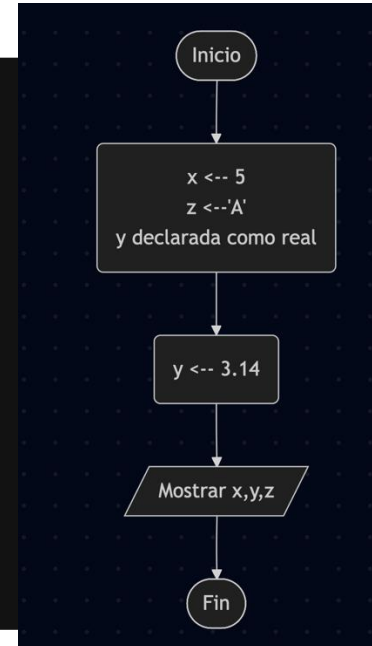
character
un carácter

character(len=N)
texto

parameter
constante

Ejemplo: declaración y asignación

```
1  program variables
2      implicit none
3      integer :: x = 5
4      real :: y
5      character :: z = 'A'
6      y = 3.14
7      print *, "x = ", x
8      print *, "y = ", y
9      print *, "z = ", z
10 end program variables
```



Lectura del código

La declaración reserva memoria y define qué clase de información puede almacenar cada variable.

```

1  program variables
2      implicit none
3      integer :: x = 5
4      real :: y
5      character :: z = 'A'
6      y = 3.14
7      print *, "x = ", x
8      print *, "y = ", y
9      print *, "z = ", z
10 end program variables

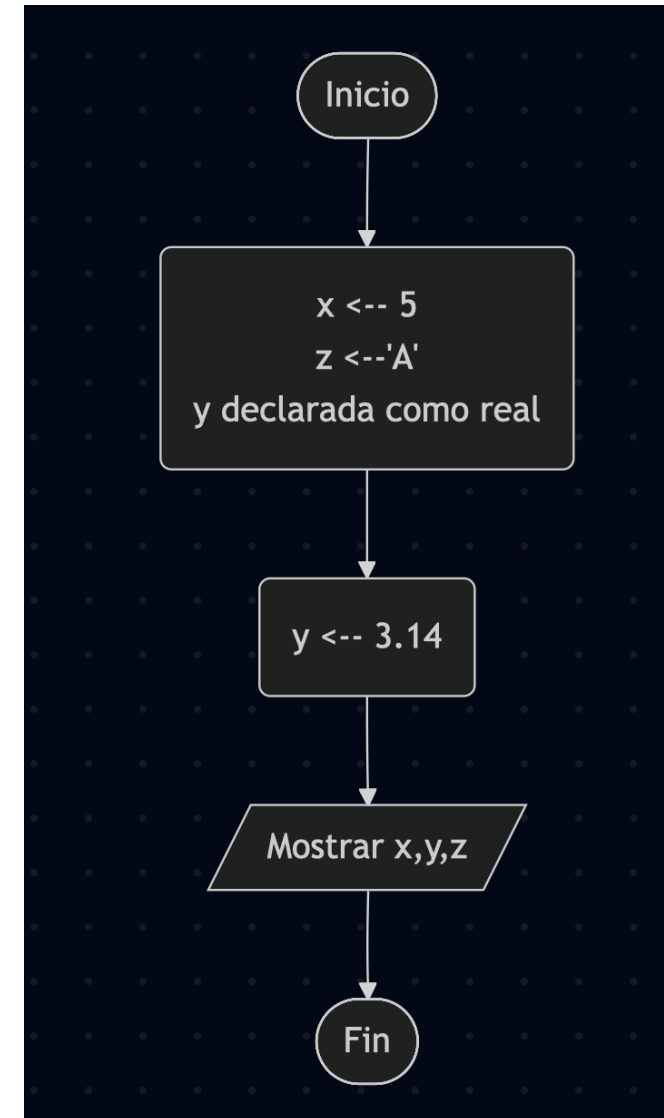
```

flowchart TD

A([Inicio]) --> B(x <-- 5
 z <--'A'
 y declarado como real)

B --> C(y <-- 3.14)

C --> D[/ Mostrar x,y,z /]

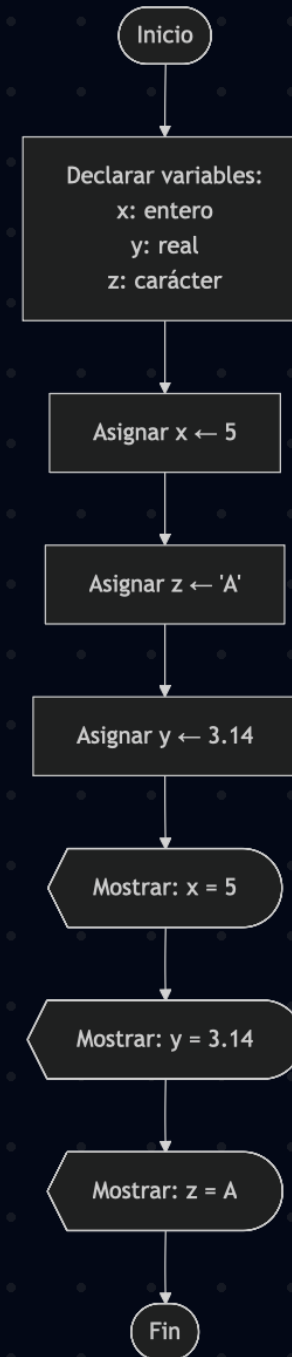


Code

Config

Docs

```
1 flowchart TD
2   A([Inicio])
3   B["Declarar variables:<br/>x: entero<br/>y: real<br/>z: carácter"]
4   C["Asignar x ← 5"]
5   D["Asignar z ← 'A'"]
6   E["Asignar y ← 3.14"]
7   F{{"Mostrar: x = 5"}}
8   G{{"Mostrar: y = 3.14"}}
9   H{{"Mostrar: z = A"}}
10  I([Fin])
11
12  A --> B
13  B --> C
14  C --> D
15  D --> E
16  E --> F
17  F --> G
18  G --> H
19  H --> I
```



Constantes, precisión y cadenas

```
1  program mas_variables
2      use iso_fortran_env, only: real64
3      implicit none
4      integer, parameter :: MAX_INTENTOS = 3
5      real(real64), parameter :: PI = 3.141592653589793_real64
6      character(len=20) :: nombre
7      logical :: aprobado = .true.
8      nombre = "Ana"
9      print *, "Nombre: ", trim(nombre)
10     print *, "PI = ", PI
11     print *, "Aprobado: ", aprobado
12 end program mas_variables
```

parameter crea constantes y real64 permite controlar explícitamente la precisión numérica.

Entrada y salida de datos

Entrada

Información que el programa recibe.

`read *, variable`

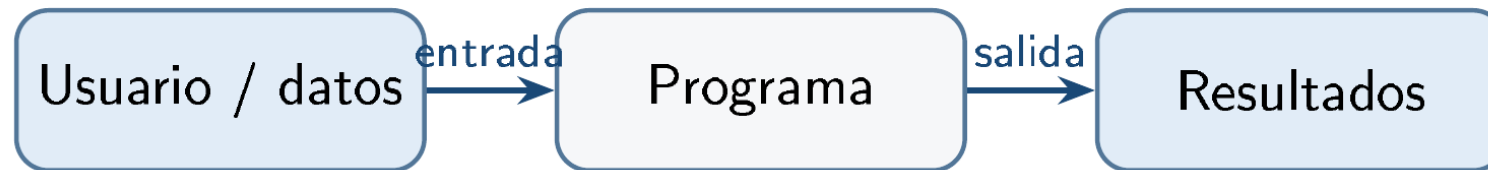
Ejemplos: teclado, archivo, sensor.

Salida

Información que el programa entrega.

`print *, resultado`

Ejemplos: pantalla, archivo, gráfica.



Ejemplo: entrada y salida

```
1 program entrada_salida
2   implicit none
3   integer :: edad
4   print *, "Ingrese su edad:"
5   read *, edad
6   print *, "En 2030 usted tendra ", edad + 4, " años."
7 end program entrada_salida
```

Observación

El programa se detiene en read hasta recibir un dato compatible con el tipo de la variable.

Ejemplo adicional: nombre y edad

```
1  program saludo
2      implicit none
3      character(len=30) :: nombre
4      integer :: edad
5      print *, "Cual es tu nombre?"
6      read '(A)', nombre
7      print *, "Cuantos años tienes?"
8      read *, edad
9      print *, "Hola ", trim(nombre), &
10         ", en 5 años tendrás ", edad + 5, " años."
11  end program saludo
```

El formato '(A)' permite leer una línea de texto que puede contener espacios.

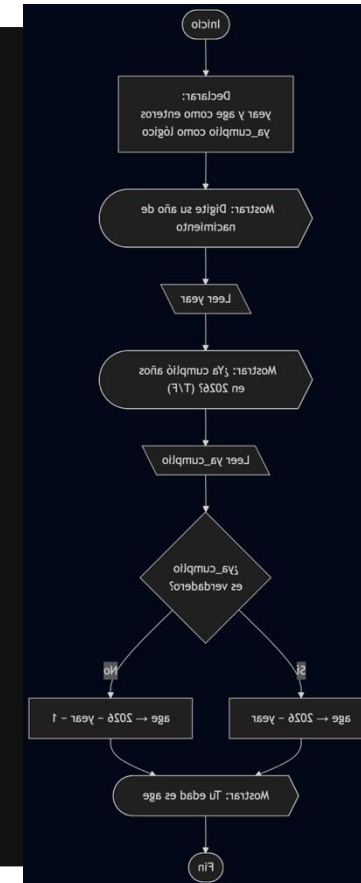
Tipo	Operadores	Resultado
Matemáticos	+ - * / **	un valor numérico
Relacionales	== /= < > <= >=	.true. o .false.
Lógicos	.and. .or. .not. .neqv.	combinación de condiciones lógicas

Ejemplo

```
dentro_rango = (x > 0.0) .and. (x < 10.0)
```

Ejemplo: cálculo de la edad

```
1  program calcular_edad
2      implicit none
3      integer :: year, age
4      logical :: ya_cumplio
5      print *, "Digite su año de nacimiento:"
6      read *, year
7      print *, "Ya cumplió años en 2026? (T/F)"
8      read *, ya_cumplio
9      if (ya_cumplio) then
10         age = 2026 - year
11     else
12         age = 2026 - year - 1
13     end if
14     print *, "Tu edad es ", age, "."
15 end program calcular_edad
```



Aquí la lógica se expresa de manera explícita con `if...then...else`, más clara para una primera lectura.

flowchart TD

A([Inicio])

B["Declarar:
year y age como enteros
ya_cumplio como lógico"]

C@{ shape: display, label: "Mostrar: Digite su año de nacimiento" }

D[/ "Leer year" /]

E@{ shape: display, label: "Mostrar: ¿Ya cumplió años en 2026? (T/F)" }

F[/ "Leer ya_cumplio" /]

G{"¿ya_cumplio
es verdadero?"}

H["age ← 2026 – year"]

I["age ← 2026 – year – 1"]

J@{ shape: display, label: "Mostrar: Tu edad es age" }

K([Fin])

A --> B

B --> C

C --> D

D --> E

E --> F

F --> G

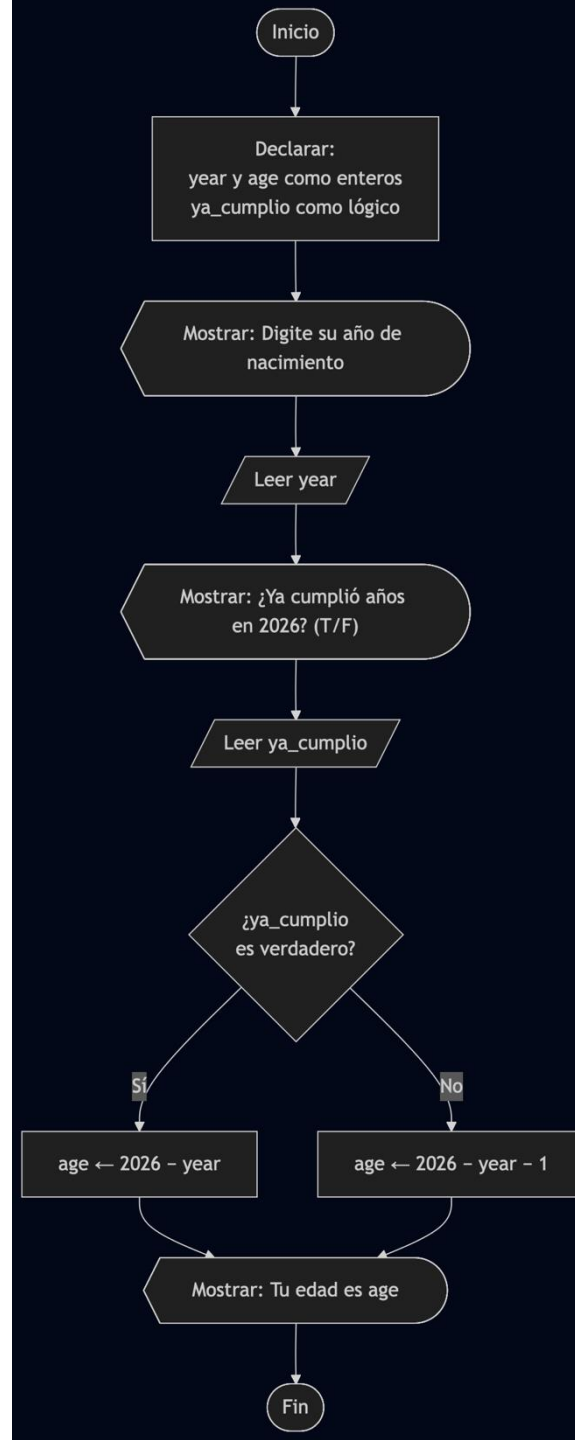
G -- "Sí" --> H

G -- "No" --> I

H --> J

I --> J

J --> K



Ejemplo adicional: operadores lógicos

```
1 program rango
2   implicit none
3   real :: x
4   logical :: dentro_rango
5   print *, "Ingrese un valor de x:"
6   read *, x
7   dentro_rango = (x > 0.0) .and. (x < 10.0)
8   print *, "x pertenece al intervalo (0,10)? ", dentro_rango
9 end program rango
```

Interpretación

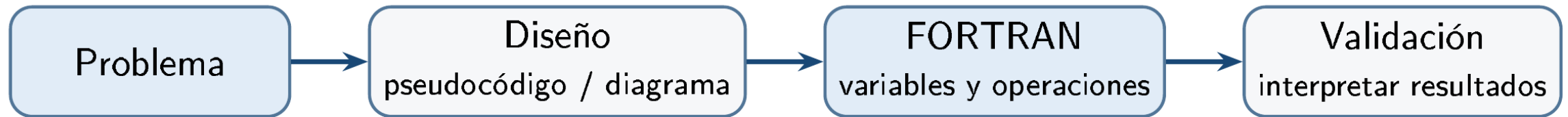
La expresión lógica será verdadera únicamente cuando ambas comparaciones sean verdaderas.

Buenas prácticas desde el primer programa

- Use siempre `implicit none`.
- Elija nombres descriptivos.
- Indente el código.
- Comente la intención, no lo evidente.
- Use constantes con `parameter`.
- Controle la precisión de los reales.
- Compile con advertencias activadas.
- Valide resultados con casos sencillos.

Compilación recomendada

```
gfortran -Wall -Wextra -fcheck=all programa.f90 -o programa
```



- Las variables tienen nombre, tipo y valor.
- read y print conectan el programa con el exterior.
- Los operadores transforman datos y permiten formular condiciones.
- La claridad del algoritmo es tan importante como la sintaxis.

1. Pseudocódigo y diagrama de flujo para cada una de las siguientes tareas (a mano o con Mermaid, como en la sección 3):

- a. Sacar el libro *“Cien años de soledad”* de la Biblioteca Central.
- b. La receta para una tortilla.
- c. Almorzar en la Cafetería Central.

2. Diseñe un programa en Fortran que calcule la altura máxima de un proyectil, conociendo el ángulo respecto a la horizontal y la rapidez inicial. Siga el proceso completo de la sección 2 y el estilo del ejemplo integrador de la sección 9 (diagrama de flujo, diccionario de datos, constantes con parameter, eco de entradas). Su reporte debe contener el diagrama de flujo y/o pseudocódigo del programa que va a realizar.

$$h_{max} = \frac{v_0^2 \sin^2(\theta)}{2g}, \quad \text{con } g = 9.8 \text{ m/s}^2.$$

- Recuerde: sin espera el ángulo en radianes (sección 7) — convierta antes de usarlo.
- Variables sugeridas: v0, angulo_grados, angulo_rad, g, h_max (todas real), y cuidado con no dividir enteros sin querer (sección 6).

Taller 1 (3/4): Celda de verificación

Celda de verificación: al final del programa, agregue un bloque que compare h_{max} contra 2–3 casos de referencia conocidos e imprima PASS/FAIL para cada uno:

Caso	angulo_grados	v0 (m/s)	h_max esperado
1	90°	9.8	4.9000 m
2	45°	20.0	10.2041 m
3	30°	20.0	5.1020 m

3. **Glosario:** defina con sus propias palabras:

algoritmo, variable, constante, tipo de dato, diagrama de flujo, pseudocódigo, entrada de datos, salida de datos, implicit none, diccionario de datos, división entera, función intrínseca, operador lógico, error de sintaxis, error de ejecución, error lógico.

Fin

Preguntas y discusión